Name: Jack Xitco Net ID: jjx221

NYU, Tandon School of Engineering
CS-1134: Data Structures and Algorithms — Spring 2019

CS-1134 – Midterm Exam

Tuesday, March 12, 2019

- You have one hour and 15 minutes.
- There are 5 questions all together, with 100 points total.
- The exam has **TWO Parts**. The first part of the exam contains this cover page and a page for scratch work. **What you write in this page will not be graded**, but you must hand it in with your exam.
- Write your Name and NetID at the top of **each page**.
- **Don't use pencils**, as they don't show up well when scanned.
- Write your answers clearly and concisely, in the spaces on the exam. Try to avoid writing near the edge of the page. **YOU MAY NOT USE THE BACKSIDE OF THE EXAM PAPERS**, as they will not be looked at. If you need extra space for an answer, use the **extra page at the end of the exam** and **mark it clearly**, so we can find it when we're grading.
- Calculators are not allowed.
- Read every question completely before answering it.
- For any questions about runtime, show the worst-case asymptotic runtime, using Theta notation.
- You do not have to do error checking. Assume all inputs to your functions are as described
- Cell phones, and any other electronic gadgets must be turned **off**.
- Do not talk to any students during the exam. If you truly do not understand what a question is asking, you may raise your hand when one of the CS1134 instructors is in the room.

Name: _____ Net ID: _____

Scratch
(This paper will not be graded)

Name: Jack Xitco Net ID: jjx221

Question 1 (20 points)

a) For each of the following, state if it is true or false (circle your choice):

I. $n^2 = \Omega(n)$ TRUE / (FALSE)

II. $\sqrt{n^2 + 7n - 5} = \theta(n)$ (TRUE) / FALSE

III. $\frac{n\sqrt{n}}{8} = O(n)$ TRUE / (FALSE)

IV. $\sqrt[4]{n} = O(log(n))$ TRUE / (FALSE)

V. $1 + 2 + 4 + 8 + 16 + \cdots + n^2 = \theta(n^2)$ (TRUE) / FALSE

b) You are given the following code:

```
def reverse_list(lst):
    rev_lst = []
    for elem in lst:
        rev_lst = [elem] + rev_lst
    return rev_lst
```

If `lst` is a list of n integers, what is the running time of `reverse_list(lst)`? Give a short explanation of your answer.

$T(n) = \theta($ n^2 $)$

The condition of the for loop garuntees n iteration calls of the loop for a list of n elements. For each iteration, two lists are added for the assignment on rev_lst, which is an $\theta(n)$ operation. Since $\theta(n)$ operation is called n times, runtime is $\theta(n^2)$

Name: Jack Xitco Net ID: jjx221

c) You are given the following code:

```
def powers_of_2_lists(n):
    i = 1
    while (i <= n):
        curr_lst = [i] * i
        print(curr_lst)
        i *= 2
    return curr_lst
```

[1] [2,2] [4,4,4,4] ...

What is the running time of `powers_of_2_lists(n)`?
Give a short explanation of your answer.

$T(n) = \theta($ log(n) $)$

the condition of the while loop appears to run n times, however, because the iterator is multiplied by 2 with each iteration of the while loop, the loop will only run $\log_2(n)$ times.

Name: Jack Xiao Net ID: jjx221

Question 2 (15 points)

a) What is printed when the following Python code is executed?

```
lst1 = [1, [2, 3]]
lst2 = [[40, 5], 60]
lst3 = lst1 + lst2
elem1 = lst1[0]
elem2 = lst1[1]

elem1 = 10
elem2[0] = 20
lst2[0][0] = 40
lst2[1] = 60

print("lst1 =", lst1)
print("lst2 =", lst2)
print("lst3 =", lst3)
```

Output:

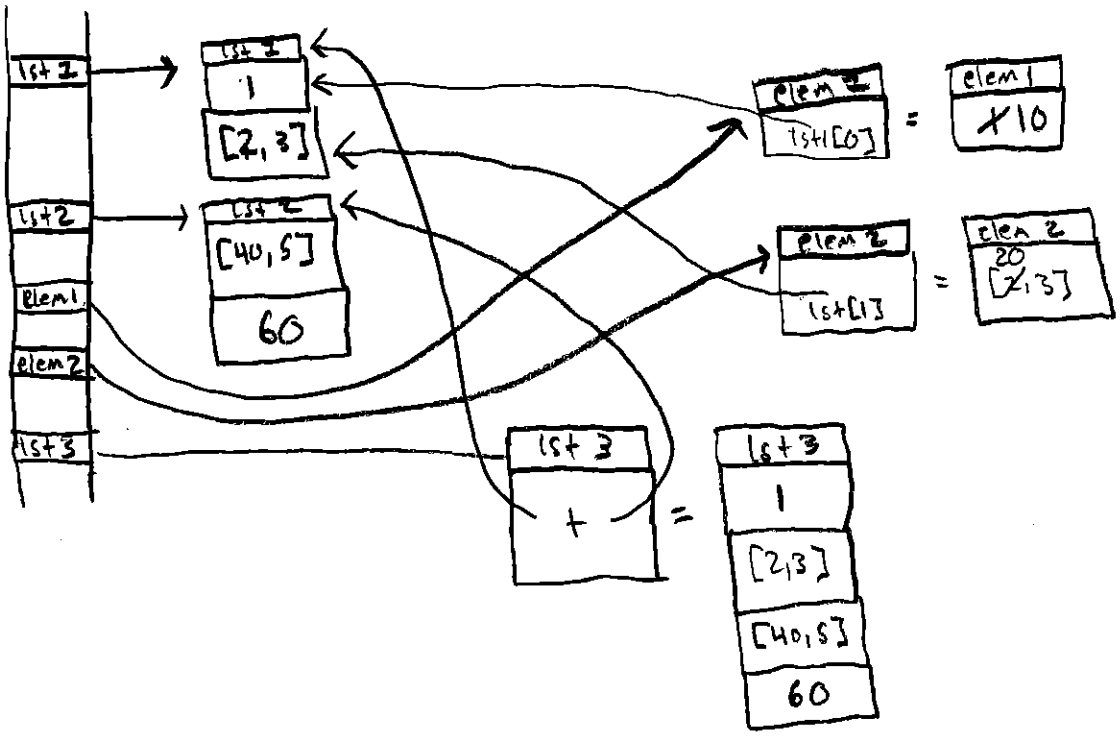
lst1 = [1,[2,3]]

lst2 = [[40,5],60]

lst3 = [1,[2,3],[40,5],60]

b) Draw the memory image at the end of the execution of the code above.

Memory image:



Name: Jack Kitco Net ID: jdk221

Question 3 (20 points):

a) Given a list of words `ls`, provide the **list comprehension** which will provide only the words which are **not** four characters:

For example, if

```
ls=['The', 'pessimist', 'complains', 'about', 'the', 'wind',
    ';', 'the', 'optimist', 'expects', 'it', 'to', 'change;',
    'the', 'realist', 'adjusts', 'the', 'sails.']
```

the call `no_four_letters_words(ls)` should return:

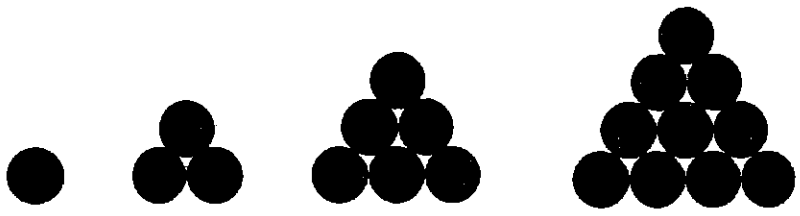
```
['The', 'pessimist', 'complains', 'about', 'the', ';', 'the',
'optimist', 'expects', 'it', 'to', 'change;', 'the',
'realist', 'adjusts', 'the', 'sails.']
```

(note, the four-letter word "wind" was removed)

```
def no_four_letters_words(ls):
    return [i for i in ls if (len(i) != 4]
```

b) Consider the following definition of a *triangular number*:
A positive integer *n* is called *a triangular number*, if you can create an equilateral triangle, by stacking *n* items.

Imagine stacking cups:



With three cups, you could create a triangle, but not with four or five (so 3 is a triangular number, but 4 and 5 are not). Six cups would be needed to create a triangle stack again, and after that 10 cups would be needed.
In short, the next amount is equal to the previous amount plus the next sequential number.
The sequence of triangular numbers goes: 1, 3, 6, 10, 15, 21, ...

Design a **generator function** which will generate the first n (a given parameter) values of the sequence, one at a time.

Name: Jack Xitco Net ID: jjx221

```
def triangular_numbers(n):
    iter = 2
    increment = 1
    for i in range(n):
        yield increment
        increment += iter
        iter += 1
```

Name: Jack Xiao Net ID: jjx221

Question 4 (25 points)
Implement the function:

```
def remove_elems_in_even_indices(lst)
```

This function gets a list, `lst`. When called, it should **mutate** `lst`, by removing all the elements that are stored in even indices. That is, `lst[0]`, `lst[2]`, `lst[4]`, ... should be removed from `lst`.

Your implementation should allow the following interaction:

```
>>> lst = [10, 20, 30, 40, 50, 60]
>>> remove_elems_in_even_indices(lst)
>>> lst
[20, 40, 60]
```

Implementation requirements:

- Your function should run in **worst case linear time**. That is, if `lst` is a list with n elements, the call `remove_elems_in_even_indices(lst)` should run in $\theta(n)$.
- Your implementation should **mutate the given list object in-place**, without using an additional data collection.

Note: Write your implementation on the next page.

Name: Jackxitco Net ID: jjx221

```
def remove_elems_in_even_indices(lst):
    temp = []
    for i in range(len(lst)-1):
        if i%2 == 1:
            temp.append(lst[i])
    lst[:] = [i for i in temp]
```

Name: Jack X14co Net ID: JJX221

Question 5 (20 points)

Consider the following definition of *opposite-pairs* in a list:
Two elements, that are at the same distance from the beginning of the list, and from the end of the list, are called an *opposite-pair*.
That is:

- The first and last elements are an opposite-pair
- The second and the second-last elements are an opposite pair
- The third and the third-last elements are an opposite pair
- etc.

Give a **recursive** implementation for the function:

```
def addative_inverse_opposite_pairs_count(lst, low, high)
```

The function is given `lst`, a list of integers, and two indices: `low` and `high` (`low` ≤ `high`), which indicate the range of indices that need to be considered.
When calling: `addative_inverse_opposite_pairs_count(lst, 0, len(lst)-1)`, it should return the number of opposite-pairs ~~in `lst`~~, in which the two numbers are additive inverses of each other.

Note that, numbers of the form x and $-x$ are considered additive inverses of each other.

For example, if `lst = [7, -6, 3, 5, 5, -3, 6, 21]`, the call `addative_inverse_opposite_pairs_count(lst, 0, 7)` should return 2 (the two opposite-pairs are (-6, 6) and (3, -3)).

Implementation requirements:

- For simplicity, you may assume that `lst` has even number of elements.
- Your function should run in **worst case linear time**. That is, if `lst` is a list with n elements, the call `addative_inverse_opposite_pairs_count(lst, 0, n-1)` should run in $\theta(n)$.
- Your function **must be recursive**.
- You are **not allowed** to:
  a) Define a helper function.
  b) Add parameters to the function's header line
  c) Set default values to the parameter.
  d) Use global variables

Hint: Your implementation does not need to use a loop.

Note: Write your implementation on the next page.

Name: Jack Xitco Net ID: jjx221

```
def addative_inverse_opposite_pairs_count(lst, low, high):
    if low >= high:
        return 0

    else:
        if lst[low] == -lst[high]:
            return 1 + addative_inverse_opposite_pairs_count(lst, low+1, high-1)
        else:
            return addative_inverse_opposite_pairs_count(lst, low+1, high-1)
```

[7, -6, 3, 5, 5, -3, 6, 11]
L L2 L3 L4 H4 H3 H2 H
H5 L5

1 + 1 + 2

Name: _____ Net ID: _____

EXTRA PAGE IF NEEDED

Note question numbers of any questions or part of questions that you are answering here.
Also, write "ANSWER IS ON LAST PAGE" near the space provided for the answer.